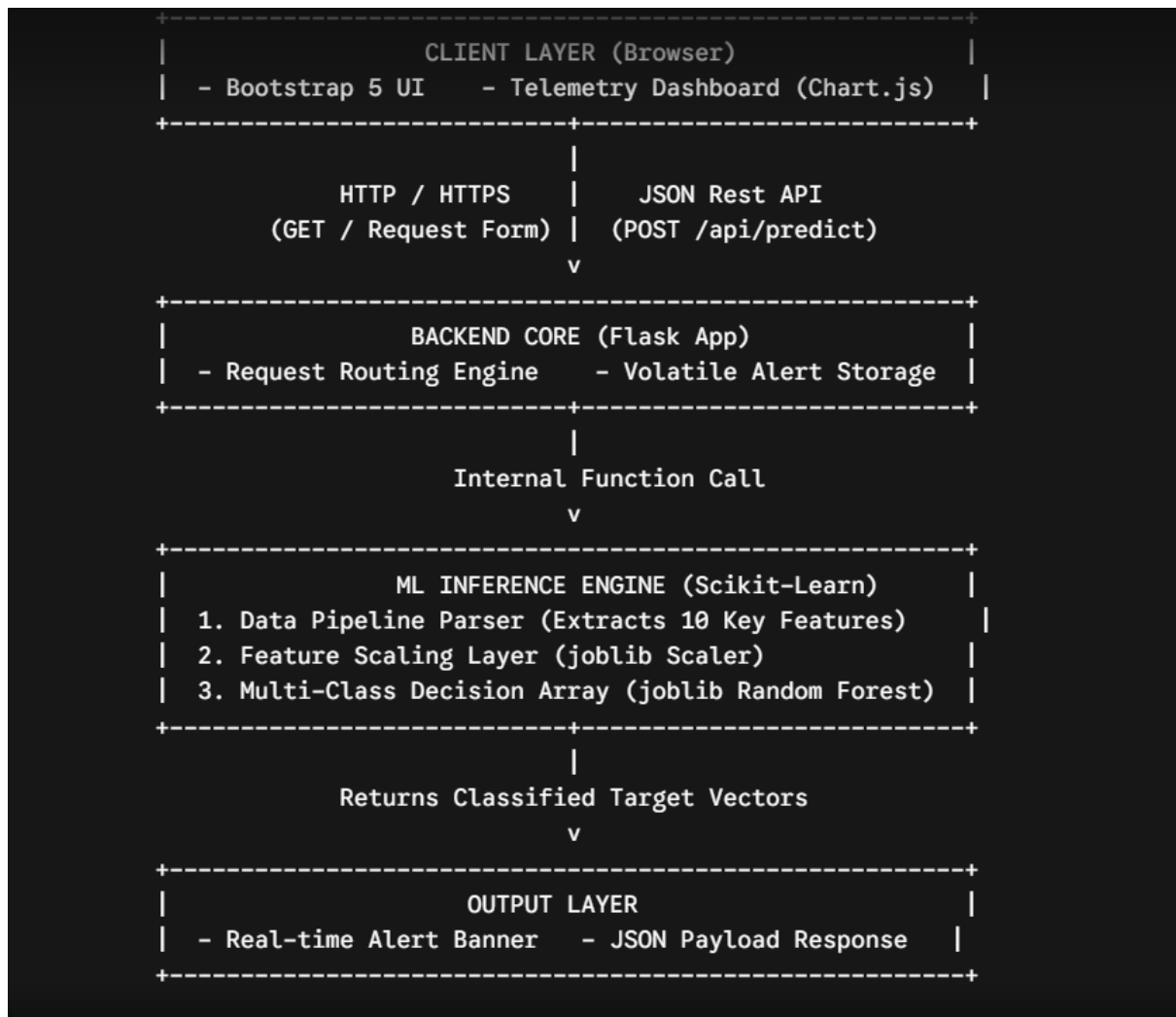


2. Solution Architecture

The solution uses a decoupled client-server architecture with an integrated Machine Learning inference layer. The web frontend communicates with the Flask server asynchronously to push telemetry, request manual predictions, or consume programmatic API features.



Here is the step-by-step breakdown of the data flow:

1. Client Layer (Browser) to Backend Core

- **Entry Points:** The data flow originates at the frontend interface, which features a **Bootstrap 5 UI** for manual inputs and a **Telemetry Dashboard (Chart.js)** for viewing statistics.
- **Transmission Protocol:** The client sends data down to the backend server using one of two communication routes:
 - **HTTP / HTTPS (GET / Request Form):** Used when a web user interacts with the standard graphical forms.

- **JSON REST API (POST /api/predict):** Used when an external application or automation script sends a structured programmatic request payload.

2. Backend Core (Flask App) Processing

- **Routing:** The **Flask App** acts as the central orchestrator. Its **Request Routing Engine** intercepts the incoming network request or API payload and directs it to the appropriate logical controller.
- **Storage Interactivity:** It manages system telemetry locally using a **Volatile Alert Storage** (in-memory caching) system to track active alerts efficiently without blocking incoming operational threads.
- **Forwarding:** Once the raw metrics are extracted from the request body, the backend handles the transition via an **Internal Function Call** directly to the underlying model execution script.

3. ML Inference Engine (Scikit-Learn)

Once the internal call executes, data moves through a strict 3-stage serialized pipeline managed by joblib:

1. **Data Pipeline Parser:** The engine isolates and extracts exactly **10 key high-signal features** from the network connection data payload.
2. **Feature Scaling Layer:** The extracted raw values are passed through a pre-trained **StandardScaler (joblib Scaler)** to apply zero-mean, unit-variance normalization, ensuring numerical parity across all feature variables.
3. **Multi-Class Decision Array:** The standardized features feed into a **Random Forest Classifier ensemble (joblib Random Forest)** consisting of 100 decision trees to execute the classification evaluation.

4. Machine Learning Engine to Output Layer

- **Vector Return:** The decision tree array completes its voting matrix and **Returns Classified Target Vectors** (the class labels and their corresponding probability confidence intervals).
- **Final Presentation & Response:** The backend intercepts the classification target results and splits the output delivery method depending on how the request entered the system:
 - **Real-time Alert Banner:** Dynamically renders the threat details and confidence bars visually onto the administrative web view.

- **JSON Payload Response:** Transmits a clean, structured JSON object containing keys like label, confidence, and is_attack directly back to the calling API terminal client.